

DSAA2043: Exercise 4

Notes

- This exercise is for practice only and does not count toward the final score.
- Both 0-indexed and 1-indexed arrays are accepted. However, when using Python, array indices must start from 0.

Problem 1. In the following binary heap:

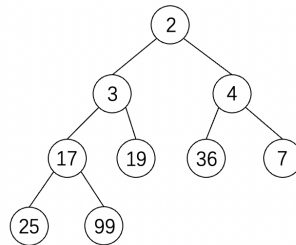


Figure 1: The exemplar binary heap

Q1.1. Show the step-by-step execution of inserting 1 into the binary heap shown in Figure 1.

Q1.2. Show the step-by-step execution of 'delete-min' for the binary heap shown in Figure 1.

Answer:

A1.1. Insert 1 at the bottom-right position, and heapify up:

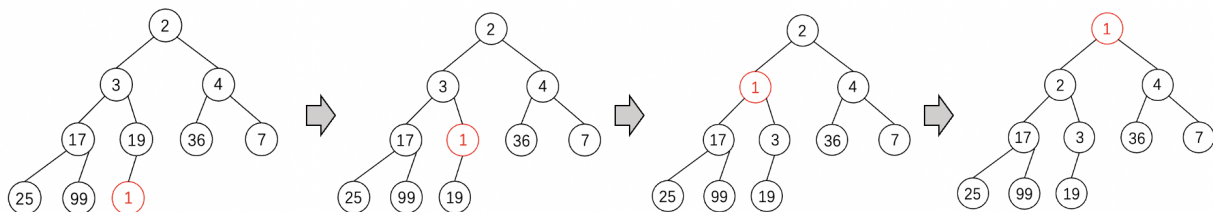


Figure 2: A1.1 visualization.

A1.2. Move the last node at the root, and heapify down:

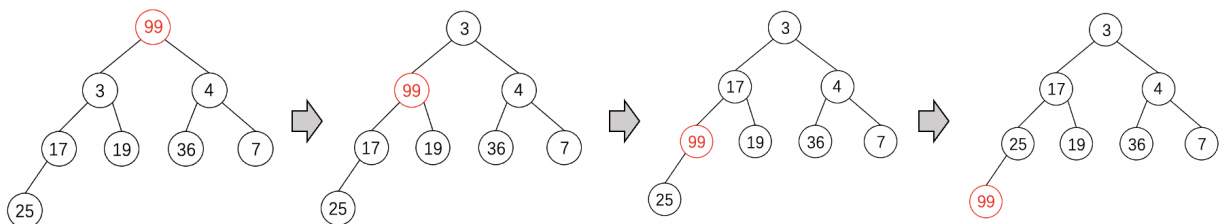


Figure 3: A1.2 visualization.

Problem 2. Suppose that we have k arrays $A_1, A_2, \dots, A_k$ of integers, such that each array has

been sorted in ascending order. Let n be the total number of integers in those arrays. Describe an algorithm to produce an array that sorts all the n integers in ascending order (you may assume that no integer exists in two arrays). Your algorithm must finish in $O(n \log k)$ time.

For example, suppose that $k = 3$, and that the three arrays are $(2, 23, 32, 35, 37)$, $(5, 10)$, and $(33, 58, 82)$. Then you should produce an array containing $(2, 5, 10, 23, 32, 33, 35, 37, 58, 82)$.

Answer: Insert the smallest element of each array into a binary heap H . This takes $O(k \log k)$ time. Then, repeat the following until H is empty:

- Perform a delete-min. Let e be the element fetched.
- Append e to the output array.
- If e comes from A_i (for some i), obtain the next element from A_i , and insert it into H . If A_i has been exhausted, then do nothing.

Each delete-min and insertion require $O(\log k)$ time because H has at most k elements. There are n delete-min and n insertions. So the total cost is $O(n \log k)$.

Problem 3. Let T be a binary tree of n nodes. For each node u of T , define its count as the number of nodes in its subtree (remember that the subtree includes the node itself). Describe an algorithm to compute the counts of all the nodes in T (you can assume that each node has reserved a memory cell for you to store the count). Your algorithm must terminate in $O(n)$ time.

Answer: We will design a recursive algorithm to perform a post order traversal of T . This algorithm at its termination will have computed the counts of all the nodes.

If the tree has only a single node, set its count to 1 and return. Otherwise, the algorithm proceeds as follows:

- Recursively compute the counts of all the nodes in the left subtree of the root.
- Recursively compute the counts of all the nodes in the right subtree of the root.
- Let c_1 be the count of the left child, and c_2 be the count of the right child. Set the count of the root to $1 + c_1 + c_2$.

To see that the running time is $O(n)$, observe that the algorithm essentially crosses each edge of the tree twice: once from the parent to the child, and another time the other way around. There are only $n-1$ edges in the tree.

Problem 4. Let T be a binary search tree (BST) of on a set S of n integers. Let x and y be two integers in S . Describe an algorithm to find the lowest common ancestor A of the nodes in T storing x and y , respectively. If A is at level ℓ (recall that the root is at level 0), your algorithm must finish in $O(1 + \ell)$ time.

Answer: Without loss of generality, suppose that $x < y$. Let v be the root of T . If the key k of v equals either x or y , return v . Otherwise:

- If $x < k < y$, report v , and finish.
- If $y < k$, set v to its left child, and repeat the above steps at (the new) v .
- Otherwise, set v to its right child, and repeat the above steps at (the new) v .

Problem 5. Prove that an insertion into the AVL-tree can trigger at most one (single/double) rotation. (How single and double rotations work are shown respectively in Figure 4a and Figure 4b).

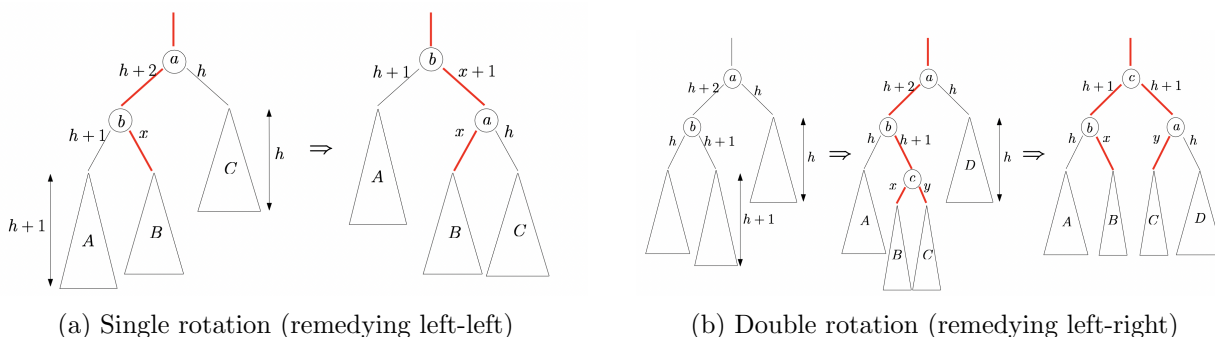


Figure 4: Rotations

Answer: Recall that each insertion descends a single root-to-leaf path. Let Π be the insertion path (the leaf on Π stores the newly inserted element). Let u be the lowest node on Π that is now imbalanced. Let p be any proper ancestor of u on Π that has become imbalanced. We will show that, fixing the imbalance at u automatically restores the balance at p .

Without loss of generality, suppose that u is in the right subtree of p . Since p is now imbalanced, currently its right subtree height must be 2 plus its left subtree height. We will prove that after fixing the imbalance at u , the right subtree height of u decreases by 1. This, in turn, decreases the right subtree height of p by 1, thus restoring the balance at p .

Let us first look at single rotation. Observe that the value of x must be h in an insertion (otherwise, a was imbalanced even before the insertion, which is impossible). This means that the height of the subtree rooted at node a decreases from $h + 2$ to $h + 1$ (note that after the rotation the subtree is rooted at node b), as claimed.

The claim is obvious in double rotation, as shown in the figure.

Problem 6. Consider the AVL-Tree below:

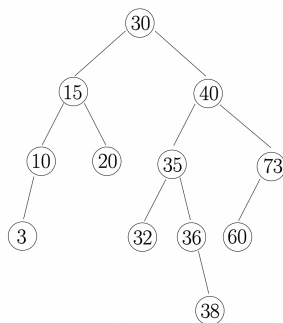


Figure 5: The exemplar AVL-Tree

Q6.1. Show the AVL-tree after inserting 37 in Figure 5.

Q6.2. Show the AVL-tree after inserting 1 in Figure 5.

Q6.3. Show the AVL-tree after deleting 60 in Figure 5.

Q6.4. Show the AVL-tree after deleting 15 in Figure 5.

Answer: The answers are shown in Figure 6.

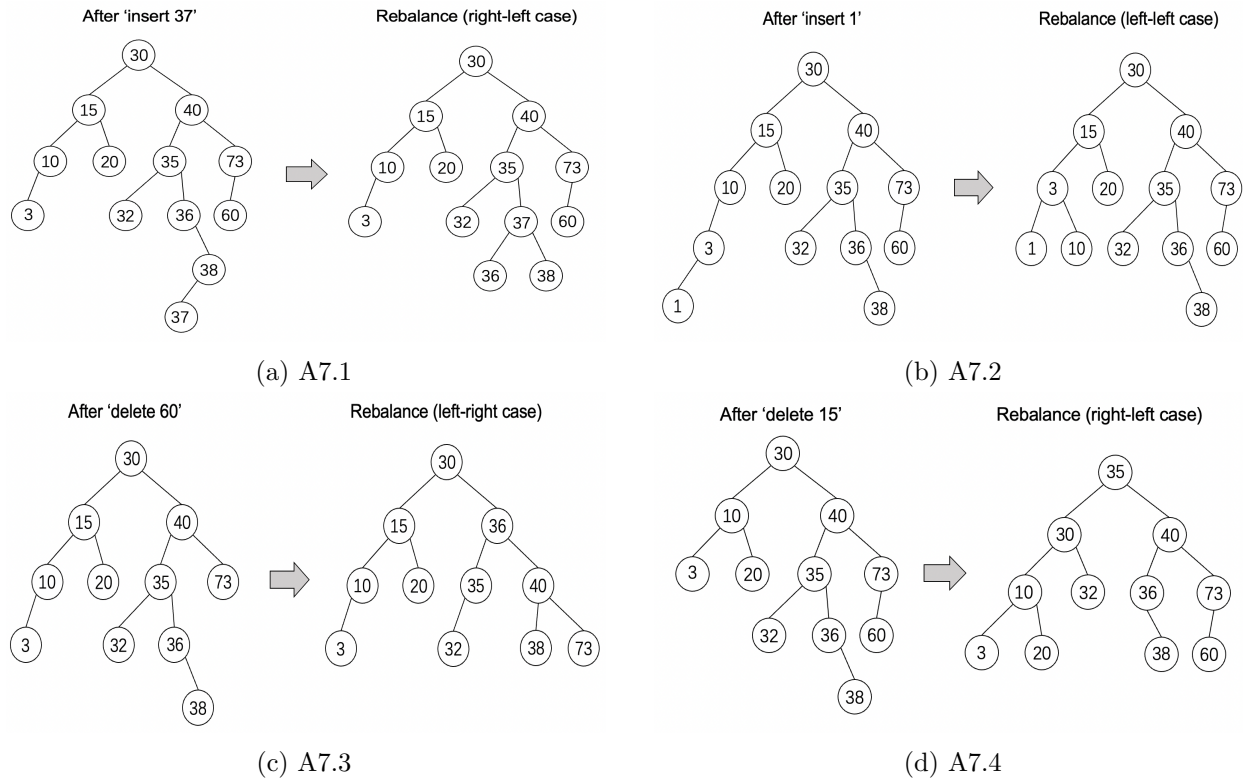


Figure 6: Answers of problem 7